JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.reflect

Class Field

java.lang.Object
    java.lang.reflect.AccessibleObject
        java.lang.reflect.Field

All Implemented Interfaces:

AnnotatedElement, Member

```
public final class Field
extends AccessibleObject
implements Member
```

A `Field` provides information about, and dynamic access to, a single field of a class or an interface. The reflected field may be a class (static) field or an instance field.

A `Field` permits widening conversions to occur during a get or set access operation, but throws an `IllegalArgumentException` if a narrowing conversion would occur.

See Also:

Member, Class, Class.getFields(), Class.getField(String), Class.getDeclaredFields(), Class.getDeclaredField(String)

Field Summary

Fields inherited from interface java.lang.reflect.Member

DECLARED, PUBLIC

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	**equals**(**Object** obj)	Compares Field agai the specifie object.
Object	**get**(**Object** obj)	Returns the value of the field represente this Field, the specifie object.
AnnotatedType	**getAnnotatedType**()	Returns an AnnotatedT object that represents use of a typ to specify t declared ty of the field represente this Field.
<T extends **Annotation**> T	**getAnnotation**(**Class**<T> annotationClass)	Returns thi element's annotation the specifie type if such annotation *present*, els null.
<T extends **Annotation**> T[]	**getAnnotationsByType**(**Class**<T> annotationClass)	Returns annotations that are *associated* with this element.
boolean	**getBoolean**(**Object** obj)	Gets the va of a static

		instance boolean fie
byte	**getByte**(**Object** obj)	Gets the va of a static o instance by field.
char	**getChar**(**Object** obj)	Gets the va of a static o instance fie of type cha of another primitive ty convertible type char v widening conversion
Annotation[]	**getDeclaredAnnotations**()	Returns annotations that are *directly present* on element.
Class<?>	**getDeclaringClass**()	Returns the Class obje representin the class or interface th declares th field represente this Field object.
double	**getDouble**(**Object** obj)	Gets the va of a static o instance fie of type dou or of anoth primitive ty convertible type doubl via a widen conversion
float	**getFloat**(**Object** obj)	Gets the va of a static o

		instance fie of type `flo` or of anoth primitive ty convertible type `float` a widening conversion
`Type`	**`getGenericType`**`()`	Returns a `T` object that represents declared ty for the field represente this `Field` object.
`int`	**`getInt`**`(`**`Object`** `obj)`	Gets the va of a static instance fie of type `int` of another primitive ty convertible type `int` vi widening conversion
`long`	**`getLong`**`(`**`Object`** `obj)`	Gets the va of a static instance fie of type `lon` of another primitive ty convertible type `long` v widening conversion
`int`	**`getModifiers`**`()`	Returns the Java langua modifiers f the field represente this `Field` object, as a integer.
`String`	**`getName`**`()`	

		Returns the name of the field represented this `Field` object.
`short`	**`getShort`**`(`**`Object`**` obj)`	Gets the value of a static or instance field of type `short` or of another primitive type convertible to type `short` via a widening conversion.
`Class``<?>`	**`getType`**`()`	Returns a `Class` object that identifies the declared type for the field represented this `Field` object.
`int`	**`hashCode`**`()`	Returns a hashcode for this `Field`.
`boolean`	**`isEnumConstant`**`()`	Returns `true` if this field represents an element of an enumerated type; returns `false` otherwise.
`boolean`	**`isSynthetic`**`()`	Returns `true` if this field is a synthetic field; returns `false` otherwise.
`void`	**`set`**`(`**`Object`**` obj, `**`Object`**` value)`	Sets the field represented

		this Field object on th specified object argument t the specifie new value.
void	**setBoolean**(**Object** obj, boolean z)	Sets the va of a field as boolean on the specifie object.
void	**setByte**(**Object** obj, byte b)	Sets the va of a field as byte on the specified object.
void	**setChar**(**Object** obj, char c)	Sets the va of a field as char on the specified object.
void	**setDouble**(**Object** obj, double d)	Sets the va of a field as double on specified object.
void	**setFloat**(**Object** obj, float f)	Sets the va of a field as float on th specified object.
void	**setInt**(**Object** obj, int i)	Sets the va of a field as a int on the specified object.

void	**setLong**(**Object** obj, long l)	Sets the va of a field as long on the specified object.
void	**setShort**(**Object** obj, short s)	Sets the va of a field as short on th specified object.
String	**toGenericString**()	Returns a string describing this Field, including it generic typ
String	**toString**()	Returns a string describing this Field.

Methods inherited from class java.lang.reflect.AccessibleObject

getAnnotations, getDeclaredAnnotation, getDeclaredAnnotationsByType, isAccessible, isAnnotationPresent, setAccessible, setAccessible

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

getDeclaringClass

```
public Class<?> getDeclaringClass()
```

Returns the Class object representing the class or interface that declares the field represented by this Field object.

Specified by:

getDeclaringClass in interface Member

Returns:

an object representing the declaring class of the underlying member

getName

```
public String getName()
```

Returns the name of the field represented by this `Field` object.

Specified by:

`getName` in interface `Member`

Returns:

the simple name of the underlying member

getModifiers

```
public int getModifiers()
```

Returns the Java language modifiers for the field represented by this `Field` object, as an integer. The `Modifier` class should be used to decode the modifiers.

Specified by:

`getModifiers` in interface `Member`

Returns:

the Java language modifiers for the underlying member

See Also:

`Modifier`

isEnumConstant

```
public boolean isEnumConstant()
```

Returns `true` if this field represents an element of an enumerated type; returns `false` otherwise.

Returns:

`true` if and only if this field represents an element of an enumerated type.

Since:

1.5

isSynthetic

```
public boolean isSynthetic()
```

Returns `true` if this field is a synthetic field; returns `false` otherwise.

Specified by:

`isSynthetic` in interface `Member`

Returns:

`true if and only if this field is a synthetic field as defined by the Java Language Specification.`

Since:

`1.5`

getType

```
public Class<?> getType()
```

Returns a `Class` object that identifies the declared type for the field represented by this `Field` object.

Returns:

`a Class object identifying the declared type of the field represented by this object`

getGenericType

```
public Type getGenericType()
```

Returns a `Type` object that represents the declared type for the field represented by this `Field` object.

If the `Type` is a parameterized type, the `Type` object returned must accurately reflect the actual type parameters used in the source code.

If the type of the underlying field is a type variable or a parameterized type, it is created. Otherwise, it is resolved.

Returns:

`a Type object that represents the declared type for the field represented by this Field object`

Throws:

`GenericSignatureFormatError - if the generic field signature does not conform to the format specified in` *`The Java™ Virtual Machine Specification`*

TypeNotPresentException - if the generic type signature of the underlying field refers to a non-existent type declaration

MalformedParameterizedTypeException - if the generic signature of the underlying field refers to a parameterized type that cannot be instantiated for any reason

Since:

1.5

equals

```
public boolean equals(Object obj)
```

Compares this Field against the specified object. Returns true if the objects are the same. Two Field objects are the same if they were declared by the same class and have the same name and type.

Overrides:

equals in class Object

Parameters:

obj - the reference object with which to compare.

Returns:

true if this object is the same as the obj argument; false otherwise.

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

Returns a hashcode for this Field. This is computed as the exclusive-or of the hashcodes for the underlying field's declaring class name and its name.

Overrides:

hashCode in class Object

Returns:

a hash code value for this object.

See Also:

Object.equals(java.lang.Object), System.identityHashCode(java.lang.Object)

toString

`public String toString()`

Returns a string describing this `Field`. The format is the access modifiers for the field, if any, followed by the field type, followed by a space, followed by the fully-qualified name of the class declaring the field, followed by a period, followed by the name of the field. For example:

```
public static final int java.lang.Thread.MIN_PRIORITY
private int java.io.FileDescriptor.fd
```

The modifiers are placed in canonical order as specified by "The Java Language Specification". This is `public`, `protected` or `private` first, and then other modifiers in the following order: `static`, `final`, `transient`, `volatile`.

Overrides:

`toString` in class `Object`

Returns:

a string describing this Field

See *The Java™ Language Specification*:

8.3.1 Field Modifiers

toGenericString

`public String toGenericString()`

Returns a string describing this `Field`, including its generic type. The format is the access modifiers for the field, if any, followed by the generic field type, followed by a space, followed by the fully-qualified name of the class declaring the field, followed by a period, followed by the name of the field.

The modifiers are placed in canonical order as specified by "The Java Language Specification". This is `public`, `protected` or `private` first, and then other modifiers in the following order: `static`, `final`, `transient`, `volatile`.

Returns:

a string describing this Field, including its generic type

Since:

1.5

See *The Java™ Language Specification*:

8.3.1 Field Modifiers

get

```
public Object get(Object obj)
           throws IllegalArgumentException,
                  IllegalAccessException
```

Returns the value of the field represented by this `Field`, on the specified object. The value is automatically wrapped in an object if it has a primitive type.

The underlying field's value is obtained as follows:

If the underlying field is a static field, the `obj` argument is ignored; it may be null.

Otherwise, the underlying field is an instance field. If the specified `obj` argument is null, the method throws a `NullPointerException`. If the specified object is not an instance of the class or interface declaring the underlying field, the method throws an `IllegalArgumentException`.

If this `Field` object is enforcing Java language access control, and the underlying field is inaccessible, the method throws an `IllegalAccessException`. If the underlying field is static, the class that declared the field is initialized if it has not already been initialized.

Otherwise, the value is retrieved from the underlying instance or static field. If the field has a primitive type, the value is wrapped in an object before being returned, otherwise it is returned as is.

If the field is hidden in the type of `obj`, the field's value is obtained according to the preceding rules.

Parameters:

`obj - object from which the represented field's value is to be extracted`

Returns:

`the value of the represented field in object obj; primitive values are wrapped in an appropriate object before being returned`

Throws:

`IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.`

`IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof).`

`NullPointerException - if the specified object is null and the field is an instance field.`

`ExceptionInInitializerError - if the initialization provoked by this method fails.`

getBoolean

```
public boolean getBoolean(Object obj)
                   throws IllegalArgumentException,
                          IllegalAccessException
```

Gets the value of a static or instance `boolean` field.

Parameters:

`obj` - the object to extract the boolean value from

Returns:

the value of the boolean field

Throws:

`IllegalAccessException` - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type boolean by a widening conversion.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`get(java.lang.Object)`

getByte

```
public byte getByte(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance `byte` field.

Parameters:

`obj` - the object to extract the byte value from

Returns:

the value of the byte field

Throws:

`IllegalAccessException` - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field

(or a subclass or implementor thereof), or if the field value cannot be converted to the type byte by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(java.lang.Object)

getChar

```
public char getChar(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance field of type char or of another primitive type convertible to type char via a widening conversion.

Parameters:

obj - the object to extract the char value from

Returns:

the value of the field converted to type char

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type char by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(java.lang.Object)

getShort

```
public short getShort(Object obj)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Gets the value of a static or instance field of type `short` or of another primitive type convertible to type `short` via a widening conversion.

Parameters:

`obj - the object to extract the short value from`

Returns:

`the value of the field converted to type short`

Throws:

`IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.`

`IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type short by a widening conversion.`

`NullPointerException - if the specified object is null and the field is an instance field.`

`ExceptionInInitializerError - if the initialization provoked by this method fails.`

See Also:

`get(java.lang.Object)`

getInt

```
public int getInt(Object obj)
           throws IllegalArgumentException,
                  IllegalAccessException
```

Gets the value of a static or instance field of type `int` or of another primitive type convertible to type `int` via a widening conversion.

Parameters:

`obj - the object to extract the int value from`

Returns:

`the value of the field converted to type int`

Throws:

`IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.`

`IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type int by a widening conversion.`

`NullPointerException - if the specified object is null and the field is an instance field.`

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(java.lang.Object)

getLong

```
public long getLong(Object obj)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Gets the value of a static or instance field of type long or of another primitive type convertible to type long via a widening conversion.

Parameters:

obj - the object to extract the long value from

Returns:

the value of the field converted to type long

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type long by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(java.lang.Object)

getFloat

```
public float getFloat(Object obj)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Gets the value of a static or instance field of type float or of another primitive type convertible to type float via a widening conversion.

Parameters:

obj - the object to extract the float value from

Returns:

the value of the field converted to type float

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type float by a widening conversion.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

get(java.lang.Object)

getDouble

```
public double getDouble(Object obj)
                 throws IllegalArgumentException,
                        IllegalAccessException
```

Gets the value of a static or instance field of type `double` or of another primitive type convertible to type `double` via a widening conversion.

Parameters:

`obj - the object to extract the double value from`

Returns:

`the value of the field converted to type double`

Throws:

`IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is inaccessible.`

`IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if the field value cannot be converted to the type double by a widening conversion.`

`NullPointerException - if the specified object is null and the field is an instance field.`

`ExceptionInInitializerError - if the initialization provoked by this method fails.`

See Also:

`get(java.lang.Object)`

set

```
public void set(Object obj,
                Object value)
         throws IllegalArgumentException,
                IllegalAccessException
```

Sets the field represented by this `Field` object on the specified object argument to the specified new value. The new value is automatically unwrapped if the underlying field has a primitive type.

The operation proceeds as follows:

If the underlying field is static, the `obj` argument is ignored; it may be null.

Otherwise the underlying field is an instance field. If the specified object argument is null, the method throws a `NullPointerException`. If the specified object argument is not an instance of the class or interface

declaring the underlying field, the method throws an `IllegalArgumentException`.

If this `Field` object is enforcing Java language access control, and the underlying field is inaccessible, the method throws an `IllegalAccessException`.

If the underlying field is final, the method throws an `IllegalAccessException` unless `setAccessible(true)` has succeeded for this `Field` object and the field is non-static. Setting a final field in this way is meaningful only during deserialization or reconstruction of instances of classes with blank final fields, before they are made available for access by other parts of a program. Use in any other context may have unpredictable effects, including cases in which other parts of a program continue to use the original value of this field.

If the underlying field is of a primitive type, an unwrapping conversion is attempted to convert the new value to a value of a primitive type. If this attempt fails, the method throws an `IllegalArgumentException`.

If, after possible unwrapping, the new value cannot be converted to the type of the underlying field by an identity or widening conversion, the method throws an `IllegalArgumentException`.

If the underlying field is static, the class that declared the field is initialized if it has not already been initialized.

The field is set to the possibly unwrapped and widened new value.

If the field is hidden in the type of `obj`, the field's value is set according to the preceding rules.

Parameters:

`obj - the object whose field should be modified`

`value - the new value for the field of obj being modified`

Throws:

`IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.`

`IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.`

`NullPointerException - if the specified object is null and the field is an instance field.`

`ExceptionInInitializerError - if the initialization provoked by this method fails.`

setBoolean

```
public void setBoolean(Object obj,
                       boolean z)
                throws IllegalArgumentException,
                       IllegalAccessException
```

Sets the value of a field as a `boolean` on the specified object. This method is equivalent to `set(obj, zObj)`, where `zObj` is a `Boolean` object and `zObj.booleanValue() == z`.

Parameters:

`obj` - the object whose field should be modified

`z` - the new value for the field of obj being modified

Throws:

`IllegalAccessException` - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`set(java.lang.Object, java.lang.Object)`

setByte

```
public void setByte(Object obj,
                    byte b)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a `byte` on the specified object. This method is equivalent to `set(obj, bObj)`, where `bObj` is a `Byte` object and `bObj.byteValue() == b`.

Parameters:

`obj` - the object whose field should be modified

`b` - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(java.lang.Object, java.lang.Object)

setChar

```
public void setChar(Object obj,
                    char c)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a `char` on the specified object. This method is equivalent to `set(obj, cObj)`, where `cObj` is a `Character` object and `cObj.charValue() == c`.

Parameters:

obj - the object whose field should be modified

c - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(java.lang.Object, java.lang.Object)

setShort

```
public void setShort(Object obj,
                     short s)
              throws IllegalArgumentException,
                     IllegalAccessException
```

Sets the value of a field as a `short` on the specified object. This method is equivalent to `set(obj, sObj)`, where `sObj` is a `Short` object and `sObj.shortValue() == s`.

Parameters:

`obj` - the object whose field should be modified

`s` - the new value for the field of obj being modified

Throws:

`IllegalAccessException` - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`set(java.lang.Object, java.lang.Object)`

setInt

```
public void setInt(Object obj,
                   int i)
            throws IllegalArgumentException,
                   IllegalAccessException
```

Sets the value of a field as an `int` on the specified object. This method is equivalent to `set(obj, iObj)`, where `iObj` is a `Integer` object and `iObj.intValue() == i`.

Parameters:

`obj` - the object whose field should be modified

`i` - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(java.lang.Object, java.lang.Object)

setLong

```
public void setLong(Object obj,
                    long l)
             throws IllegalArgumentException,
                    IllegalAccessException
```

Sets the value of a field as a `long` on the specified object. This method is equivalent to `set(obj, lObj)`, where `lObj` is a `Long` object and `lObj.longValue() == l`.

Parameters:

`obj` - the object whose field should be modified

`l` - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(java.lang.Object, java.lang.Object)

setFloat

```
public void setFloat(Object obj,
                     float f)
              throws IllegalArgumentException,
                     IllegalAccessException
```

Sets the value of a field as a `float` on the specified object. This method is equivalent to `set(obj, fObj)`, where `fObj` is a `Float` object and `fObj.floatValue() == f`.

Parameters:

`obj` - the object whose field should be modified

`f` - the new value for the field of obj being modified

Throws:

`IllegalAccessException` - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

`IllegalArgumentException` - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

`NullPointerException` - if the specified object is null and the field is an instance field.

`ExceptionInInitializerError` - if the initialization provoked by this method fails.

See Also:

`set(java.lang.Object, java.lang.Object)`

setDouble

```
public void setDouble(Object obj,
                      double d)
               throws IllegalArgumentException,
                      IllegalAccessException
```

Sets the value of a field as a `double` on the specified object. This method is equivalent to `set(obj, dObj)`, where `dObj` is a `Double` object and `dObj.doubleValue() == d`.

Parameters:

`obj` - the object whose field should be modified

`d` - the new value for the field of obj being modified

Throws:

IllegalAccessException - if this Field object is enforcing Java language access control and the underlying field is either inaccessible or final.

IllegalArgumentException - if the specified object is not an instance of the class or interface declaring the underlying field (or a subclass or implementor thereof), or if an unwrapping conversion fails.

NullPointerException - if the specified object is null and the field is an instance field.

ExceptionInInitializerError - if the initialization provoked by this method fails.

See Also:

set(java.lang.Object, java.lang.Object)

getAnnotation

```
public <T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Description copied from interface: AnnotatedElement

Returns this element's annotation for the specified type if such an annotation is *present,* else null.

Specified by:

getAnnotation in interface AnnotatedElement

Overrides:

getAnnotation in class AccessibleObject

Type Parameters:

T - the type of the annotation to query for and return if present

Parameters:

annotationClass - the Class object corresponding to the annotation type

Returns:

this element's annotation for the specified annotation type if present on this element, else null

Throws:

NullPointerException - if the given annotation class is null

Since:

1.5

getAnnotationsByType

```
public <T extends Annotation> T[] getAnnotationsByType(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `AnnotatedElement.getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS 9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getAnnotationsByType` in interface `AnnotatedElement`

Overrides:

`getAnnotationsByType` in class `AccessibleObject`

Type Parameters:

`T` - the type of the annotation to query for and return if present

Parameters:

`annotationClass` - the Class object corresponding to the annotation type

Returns:

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

Throws:

`NullPointerException` - if the given annotation class is null

Since:

1.8

getDeclaredAnnotations

```
public Annotation[] getDeclaredAnnotations()
```

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

Specified by:

`getDeclaredAnnotations` in interface `AnnotatedElement`

Overrides:

`getDeclaredAnnotations` in class `AccessibleObject`

Returns:

annotations directly present on this element

getAnnotatedType

```
public AnnotatedType getAnnotatedType()
```

Returns an AnnotatedType object that represents the use of a type to specify the declared type of the field represented by this Field.

Returns:

an object representing the declared type of the field represented by this Field

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.